

CS 203: Software Tools & Techniques for AI

Lab Assignment -7

Team - 06

- Swayam Borate
- Aditya Borate

Github: https://github.com/adi776borate/CS203-STT-Labs/tree/main/CS203_Lab_07

Kaggle Dataset: <https://www.kaggle.com/datasets/beweird/llama-embeddings/data>

- Version

```
Python implementation: CPython
Python version       : 3.10.12
IPython version      : 7.34.0
```

```
numpy      : 1.26.4
pandas     : 2.2.3
matplotlib : 3.7.5
seaborn    : 0.12.2
torch      : 2.5.1+cu121
sklearn    : 1.2.2
transformers: 4.47.0
```

```
Compiler   : GCC 11.4.0
OS         : Linux
Release    : 6.6.56+
Machine    : x86_64
Processor  : x86_64
CPU cores  : 4
Architecture: 64bit
```

- Model Architecture

```
class MLPModel(nn.Module):
    def __init__(self, input_size=10000, hidden_sizes=[512, 256, 128, 64], output_size=2):
        super(MLPModel, self).__init__()

        # Define layers
        self.fc1 = nn.Linear(input_size, hidden_sizes[0])
        self.fc2 = nn.Linear(hidden_sizes[0], hidden_sizes[1])
        self.fc3 = nn.Linear(hidden_sizes[1], hidden_sizes[2])
        self.fc4 = nn.Linear(hidden_sizes[2], hidden_sizes[3])
        self.fc5 = nn.Linear(hidden_sizes[3], output_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3) # Dropout for regularization

    def forward(self, x):
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.dropout(self.relu(self.fc2(x)))
        x = self.dropout(self.relu(self.fc3(x)))
        x = self.dropout(self.relu(self.fc4(x)))
        x = self.fc5(x)
        return x
```

- Model Summary

```
=====
Layer (type:depth-idx)      Input Shape      Output Shape      Param #           Trainable
=====
MLPModel                    [1, 10000]       [1, 2]            --                True
├─Linear: 1-1                [1, 10000]       [1, 512]          5,120,512         True
├─ReLU: 1-2                  [1, 512]         [1, 512]          --                --
├─Dropout: 1-3                [1, 512]         [1, 512]          --                --
├─Linear: 1-4                 [1, 512]         [1, 256]          131,328           True
├─ReLU: 1-5                  [1, 256]         [1, 256]          --                --
├─Dropout: 1-6                [1, 256]         [1, 256]          --                --
├─Linear: 1-7                 [1, 256]         [1, 128]          32,896            True
├─ReLU: 1-8                  [1, 128]         [1, 128]          --                --
├─Dropout: 1-9                [1, 128]         [1, 128]          --                --
├─Linear: 1-10                [1, 128]         [1, 64]           8,256             True
├─ReLU: 1-11                  [1, 64]          [1, 64]           --                --
├─Dropout: 1-12               [1, 64]          [1, 64]           --                --
├─Linear: 1-13                [1, 64]          [1, 2]            130               True
=====
Total params: 5,293,122
Trainable params: 5,293,122
Non-trainable params: 0
Total mult-adds (M): 5.29
=====
Input size (MB): 0.04
Forward/backward pass size (MB): 0.01
Params size (MB): 21.17
Estimated Total Size (MB): 21.22
=====
```

- Hyperparameters

```
# Hyperparameters
HYPERPARAMS = {
    "input_size": 10000,
    "hidden_sizes": [512, 256, 128, 64],
    "output_size": 2,
    "epochs": 10,
    "batch_size": 32,
    "learning_rate": 0.0001
}
```

- Logged metrics

→ Training the MLP model on bag of Words on Dataset-1 (Saved the model-checkpoint as zip (mlp_bow.gz))

```
# Initialize model
model_bow = MLPModel(input_size=10000)

# Train the model
trained_model_bow = train_model(model_bow, X_train_bow, y_train, X_val_bow, y_val, checkpoint_path="mlp_bow", model_type="bow")

Epoch 1: Train Loss = 0.6917, Val Loss = 0.6896, Val Accuracy = 0.5152
Epoch 2: Train Loss = 0.6074, Val Loss = 0.4816, Val Accuracy = 0.7890
Epoch 3: Train Loss = 0.3045, Val Loss = 0.4525, Val Accuracy = 0.8049
Epoch 4: Train Loss = 0.1319, Val Loss = 0.5480, Val Accuracy = 0.7991
Epoch 5: Train Loss = 0.0604, Val Loss = 0.6543, Val Accuracy = 0.7955
Epoch 6: Train Loss = 0.0312, Val Loss = 0.7662, Val Accuracy = 0.7962
Epoch 7: Train Loss = 0.0177, Val Loss = 0.9278, Val Accuracy = 0.7984
Epoch 8: Train Loss = 0.0103, Val Loss = 0.9484, Val Accuracy = 0.8020
Epoch 9: Train Loss = 0.0056, Val Loss = 1.0561, Val Accuracy = 0.7977
Epoch 10: Train Loss = 0.0040, Val Loss = 1.1485, Val Accuracy = 0.7955
Compressed checkpoint saved at: mlp_bow.gz
Best model saved with Validation Accuracy: 0.8049
```

→ Training the MLP model on llama embeddings on Dataset-1 (Saved the model-checkpoint as zip (mlp_llama.gz))

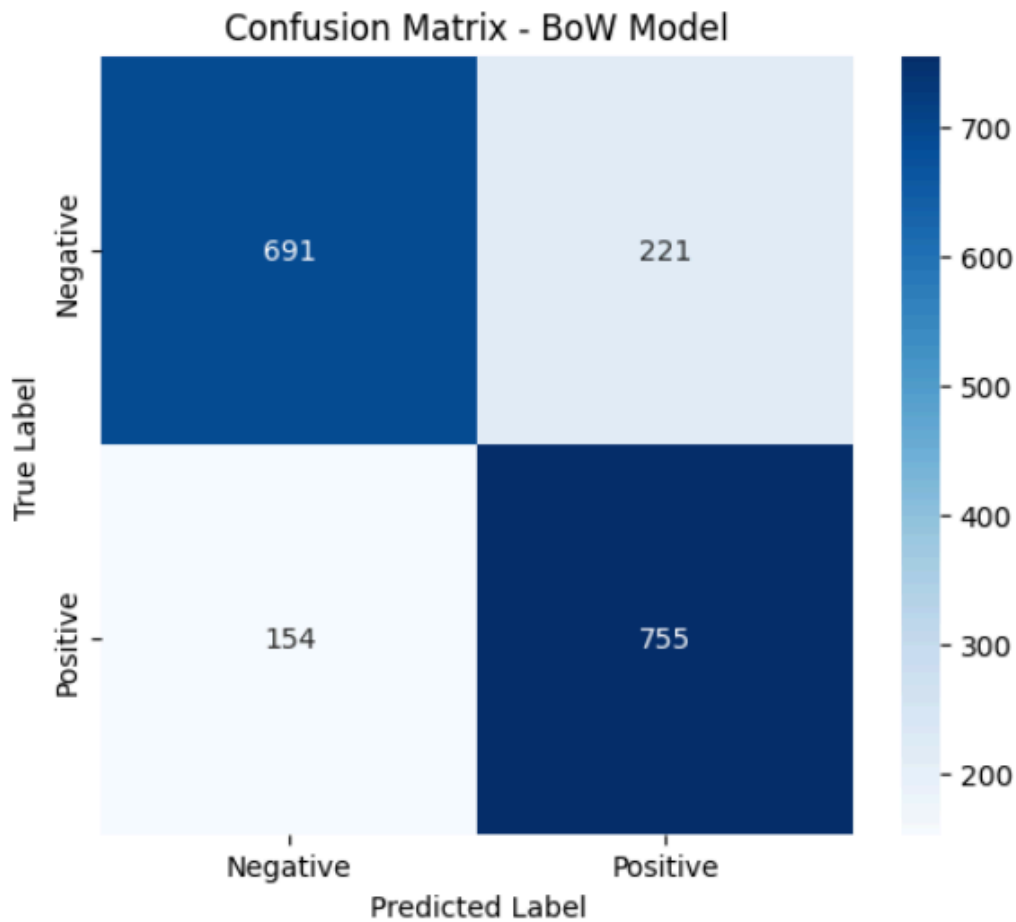
```
# Initialize model
model_llama = MLPModel(input_size=X_train_llama.shape[1])

# Train the model
trained_model_llama = train_model(model_llama, X_train_llama, y_train, X_val_llama, y_val, checkpoint_path="mlp_llama", model_type="llama")
```

Epoch 1: Train Loss = 0.4950, Val Loss = 0.3412, Val Accuracy = 0.8475
Epoch 2: Train Loss = 0.3330, Val Loss = 0.3137, Val Accuracy = 0.8728
Epoch 3: Train Loss = 0.2950, Val Loss = 0.2929, Val Accuracy = 0.8801
Epoch 4: Train Loss = 0.2643, Val Loss = 0.2675, Val Accuracy = 0.8931
Epoch 5: Train Loss = 0.2473, Val Loss = 0.2523, Val Accuracy = 0.8952
Epoch 6: Train Loss = 0.2195, Val Loss = 0.2637, Val Accuracy = 0.9017
Epoch 7: Train Loss = 0.2109, Val Loss = 0.2749, Val Accuracy = 0.8923
Epoch 8: Train Loss = 0.1912, Val Loss = 0.2682, Val Accuracy = 0.8988
Epoch 9: Train Loss = 0.1687, Val Loss = 0.2597, Val Accuracy = 0.9053
Epoch 10: Train Loss = 0.1732, Val Loss = 0.2557, Val Accuracy = 0.9068
Compressed checkpoint saved at: mlp_llama.gz
Best model saved with Validation Accuracy: 0.9068

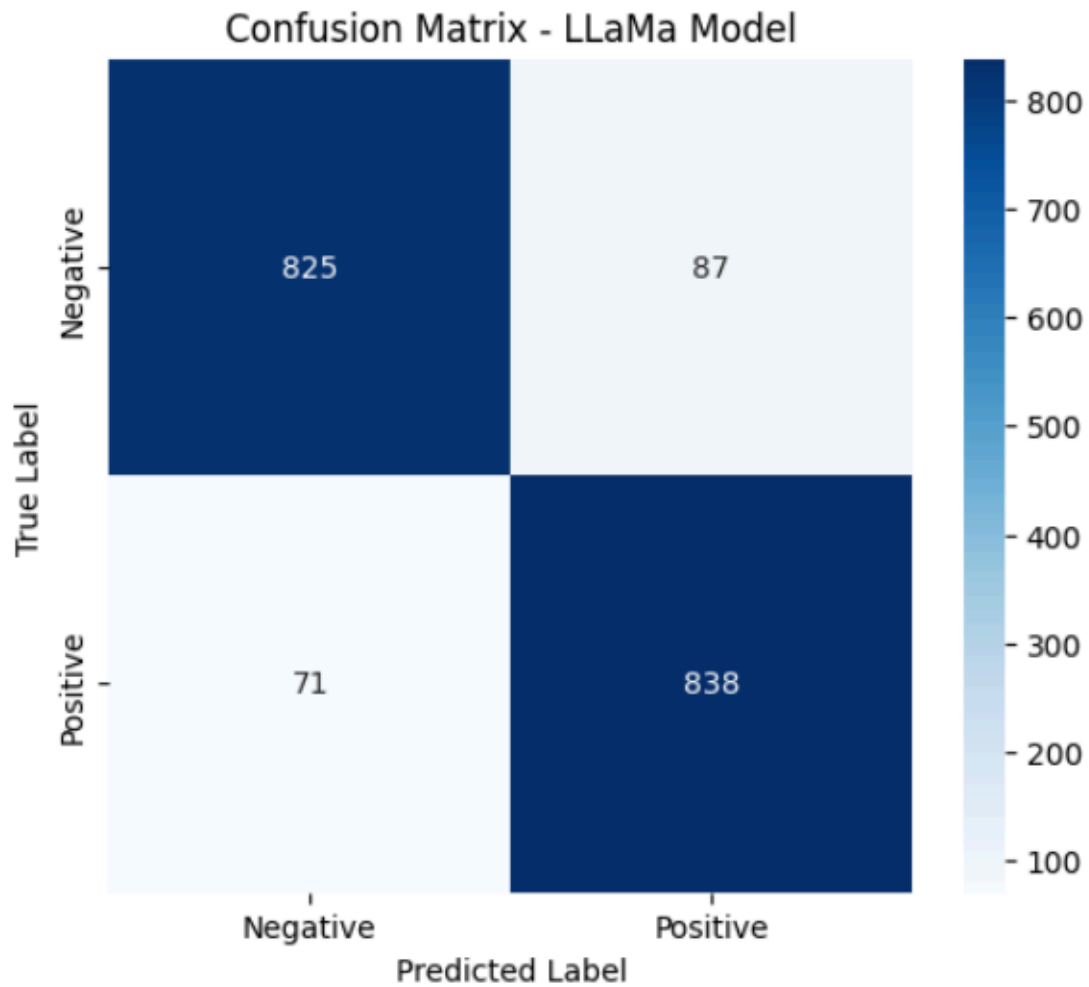
→ Predictions of the MLP model trained on Bag of words (Dataset-1)

Confusion Matrix - BoW Model - Test Accuracy: 0.7941



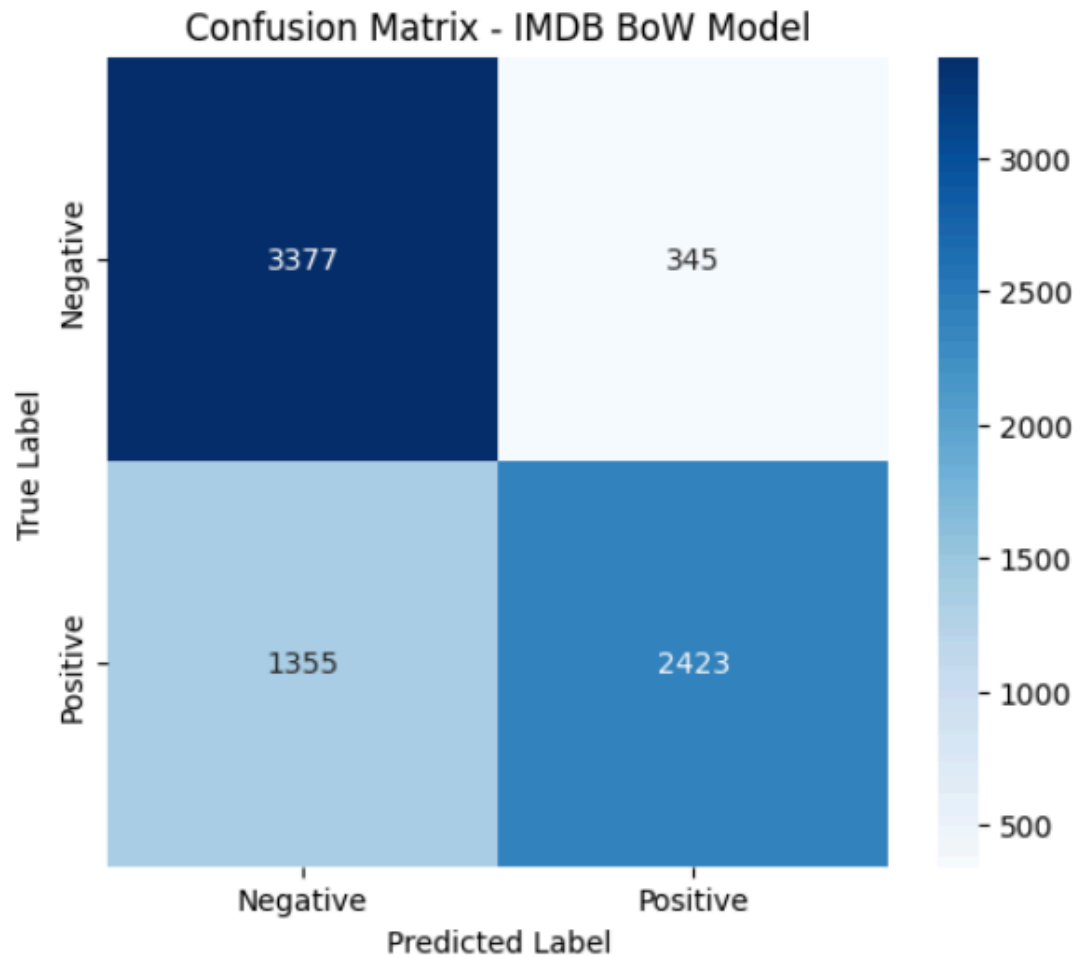
→ Predictions of the MLP model trained on llama embeddings (Dataset-1)

Confusion Matrix - LLaMa Model - Test Accuracy: 0.9132



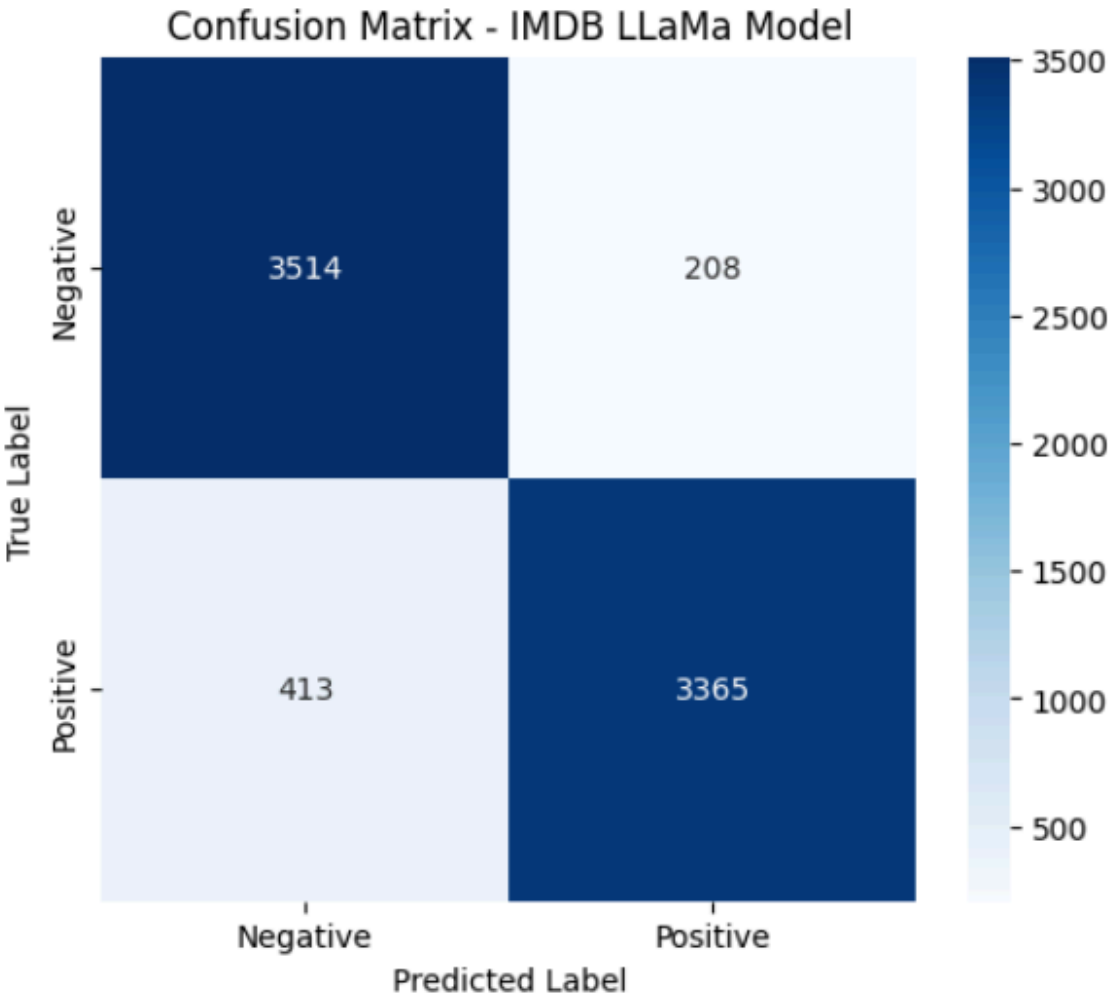
→ Predictions of the MLP model on Bag of words (Dataset-2)

Confusion Matrix - IMDB BoW Model - Test Accuracy: 0.7733



→ Predictions of the llama model on llama embeddings (Dataset-2)

Confusion Matrix - IMDB LLaMa Model - Test Accuracy: 0.9172



- Loading the saved models to further train on Dataset-2 (model_bow_z is the model with the best accuracy which was trained on the bag of words Dataset-1) (model_llama_z is the model with the best accuracy which was trained on the llama embeddings Dataset-1)

```
# Initialize empty model structures
model_bow_z = MLPModel()
model_llama_z = MLPModel(input_size=X_train_llama.shape[1])

# Load saved models from compressed checkpoints
model_bow_z = load_compressed_checkpoint("mlp_bow.gz", model_bow)
model_llama_z = load_compressed_checkpoint("mlp_llama.gz", model_llama)

# Move models to device
device = "cuda" if torch.cuda.is_available() else "cpu"
model_bow_z.to(device)
model_llama_z.to(device)

print("Checkpoint models loaded successfully and ready for further training!")
```

Model loaded from: mlp_bow.gz
Hyperparameters: {'input_size': 10000, 'hidden_sizes': [512, 256, 128, 64], 'output_size': 2, 'epochs': 10, 'batch_size': 32, 'learning_rate': 0.0001}
Final Accuracy Before Resume: 0.8049

Model loaded from: mlp_llama.gz
Hyperparameters: {'input_size': 10000, 'hidden_sizes': [512, 256, 128, 64], 'output_size': 2, 'epochs': 10, 'batch_size': 32, 'learning_rate': 0.0001}
Final Accuracy Before Resume: 0.9068

Checkpoint models loaded successfully and ready for further training!

→ Training the saved bag of words model on Dataset-2 (saving the final model with best accuracy)

```
trained_model_bow_imdb = resume_training(model_bow_z, X_train_imdb_bow, y_train_imdb, X_val_imdb_bow, y_val_imdb, checkpoint_path="mlp_bow_2", model_type="bow_2")
```

Resuming training with mlp_bow_2 on IMDB dataset...

Epoch	Train Loss	Val Loss	Val Accuracy
1	0.0193	0.6318	0.8804
2	0.0092	0.7467	0.8775
3	0.0084	0.7339	0.8808
4	0.0090	0.7281	0.8789
5	0.0047	0.8115	0.8768
6	0.0059	0.7736	0.8839
7	0.0066	0.7873	0.8784
8	0.0057	0.7953	0.8763
9	0.0040	0.9794	0.8803
10	0.0063	0.7326	0.8813

Compressed checkpoint saved at: mlp_bow_2_imdb.gz
Best model saved with Validation Accuracy: 0.8839

→ Training the saved llama embeddings model on Dataset-2 (also saving the final model with best accuracy)

```
trained_model_llama_imdb = resume_training(model_llama_z, X_train_imdb_llama, y_train_imdb, X_val_imdb_llama, y_val_imdb, checkpoint_path="mlp_llama_2", model_type="llama_2")
```

Resuming training with mlp_llama_2 on IMDB dataset...

Epoch	Train Loss	Val Loss	Val Accuracy
1	0.0914	0.1535	0.9520
2	0.0841	0.1549	0.9535
3	0.0784	0.1637	0.9529
4	0.0770	0.1657	0.9480
5	0.0708	0.1667	0.9511
6	0.0679	0.1642	0.9539
7	0.0627	0.1661	0.9539
8	0.0597	0.1717	0.9515
9	0.0550	0.1948	0.9496
10	0.0547	0.2050	0.9511

Compressed checkpoint saved at: mlp_llama_2_imdb.gz
Best model saved with Validation Accuracy: 0.9539

- Final Results (Validation loss & Accuracy on Dataset-1 (original) and Dataset -2 (IMDB))

```
# Compute validation loss & accuracy on Dataset 1 (original) and IMDB
evaluate_model(trained_model_bow_imdb, X_val_bow, y_val, dataset_name="Dataset 1 - BoW")
evaluate_model(trained_model_bow_imdb, X_val_imdb_bow, y_val_imdb, dataset_name="IMDB - BoW")

evaluate_model(trained_model_llama_imdb, X_val_llama, y_val, dataset_name="Dataset 1 - LLaMa")
evaluate_model(trained_model_llama_imdb, X_val_imdb_llama, y_val_imdb, dataset_name="IMDB - LLaMa")
```

Validation Results for Dataset 1 - BoW:

Loss: 1.1242

Accuracy: 0.8042

Validation Results for IMDB - BoW:

Loss: 0.7303

Accuracy: 0.8813

Validation Results for Dataset 1 - LLaMa:

Loss: 1.2145

Accuracy: 0.8367

Validation Results for IMDB - LLaMa:

Loss: 0.2054

Accuracy: 0.9511

3... (0.20542030036449432, 0.9510666666666666)

- Final Results (Final test accuracy on Dataset-1)

```
# Compute final test accuracy on Dataset-1
evaluate_model(trained_model_bow_imdb, X_test_bow, y_test, dataset_name="Dataset 1 - BoW")
evaluate_model(trained_model_llama_imdb, X_test_llama, y_test, dataset_name="Dataset 1 - LLaMa")
```

Validation Results for Dataset 1 - BoW:

Loss: 1.1707

Accuracy: 0.7814

Validation Results for Dataset 1 - LLaMa:

Loss: 1.0337

Accuracy: 0.8517

(1.0337028503417969, 0.8517298187808896)

- Final Results (Final test accuracy on IMDB dataset)

```
# Compute final test accuracy on IMDB dataset
evaluate_model(trained_model_bow_imdb, X_test_imdb_bow, y_test_imdb, dataset_name="IMDB Test - BoW")
evaluate_model(trained_model_llama_imdb, X_test_imdb_llama, y_test_imdb, dataset_name="IMDB Test - LLaMa")
```

Validation Results for IMDB Test - BoW:

Loss: 0.7521

Accuracy: 0.8865

Validation Results for IMDB Test - LLaMa:

Loss: 0.2119

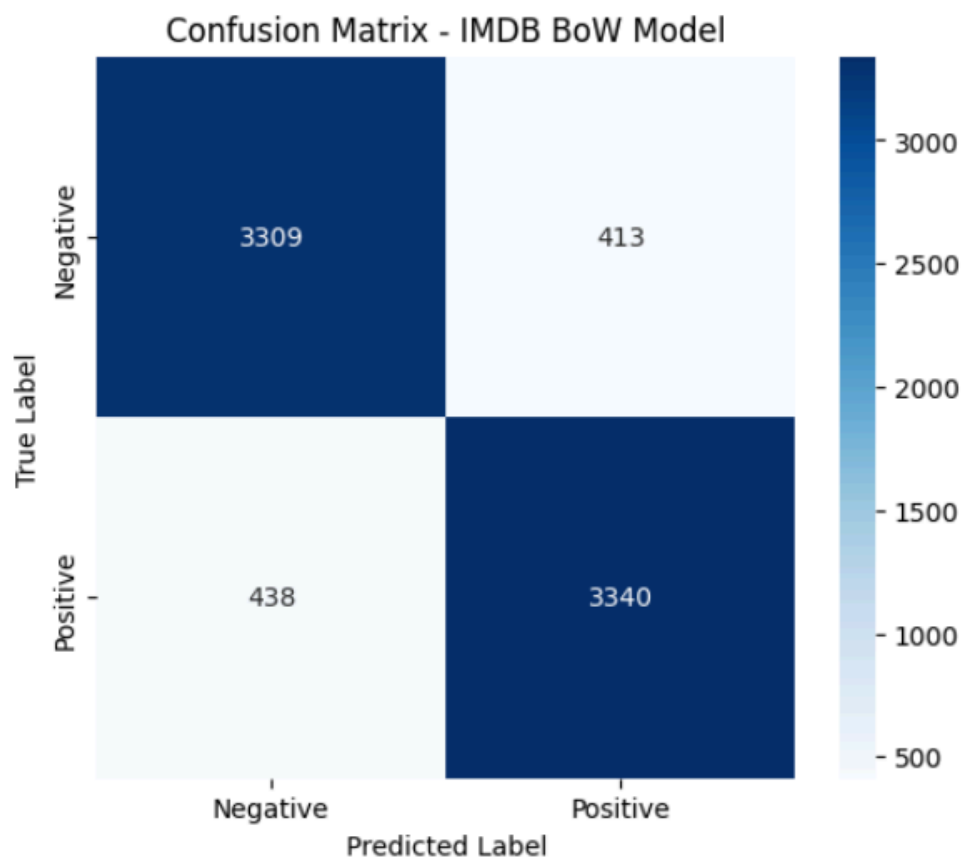
Accuracy: 0.9485

(0.21189169585704803, 0.9485333333333333)

- Predictions of the final model (Bag of words), trained on both datasets, on the IMDB dataset.

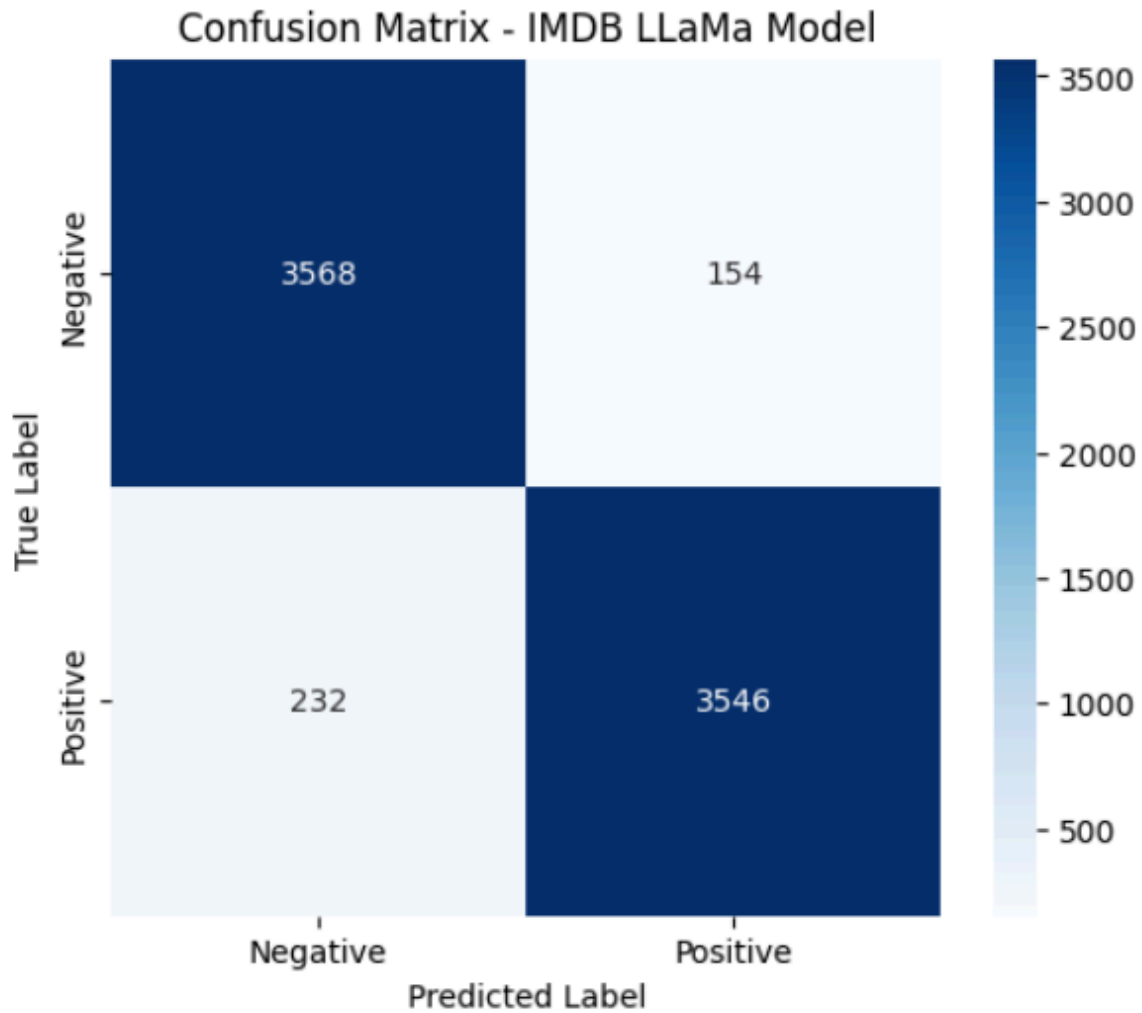
```
# Plot confusion matrices for final IMDB test set
plot_confusion_matrix(trained_model_bow_imdb, X_test_imdb_bow, y_test_imdb, title="Confusion Matrix - IMDB BoW Model")
plot_confusion_matrix(trained_model_llama_imdb, X_test_imdb_llama, y_test_imdb, title="Confusion Matrix - IMDB LLaMa Model")
```

Confusion Matrix - IMDB BoW Model - Test Accuracy: 0.8865

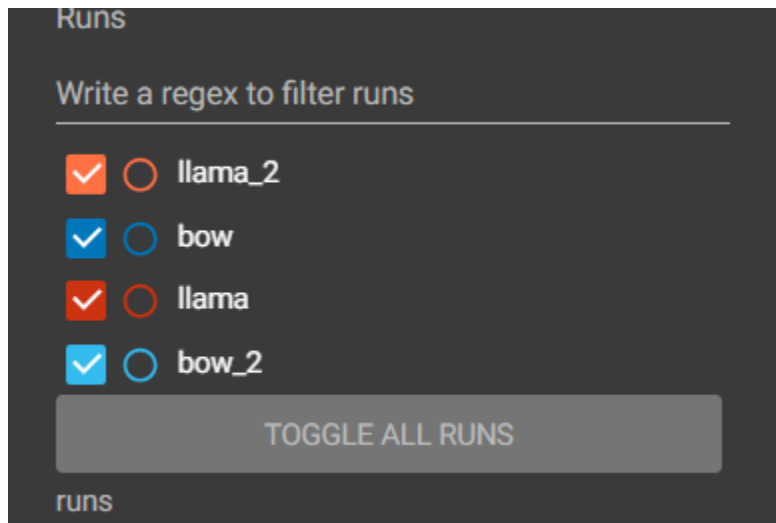


- Predictions of the final model (llama model), trained on both datasets, on the IMDB dataset.

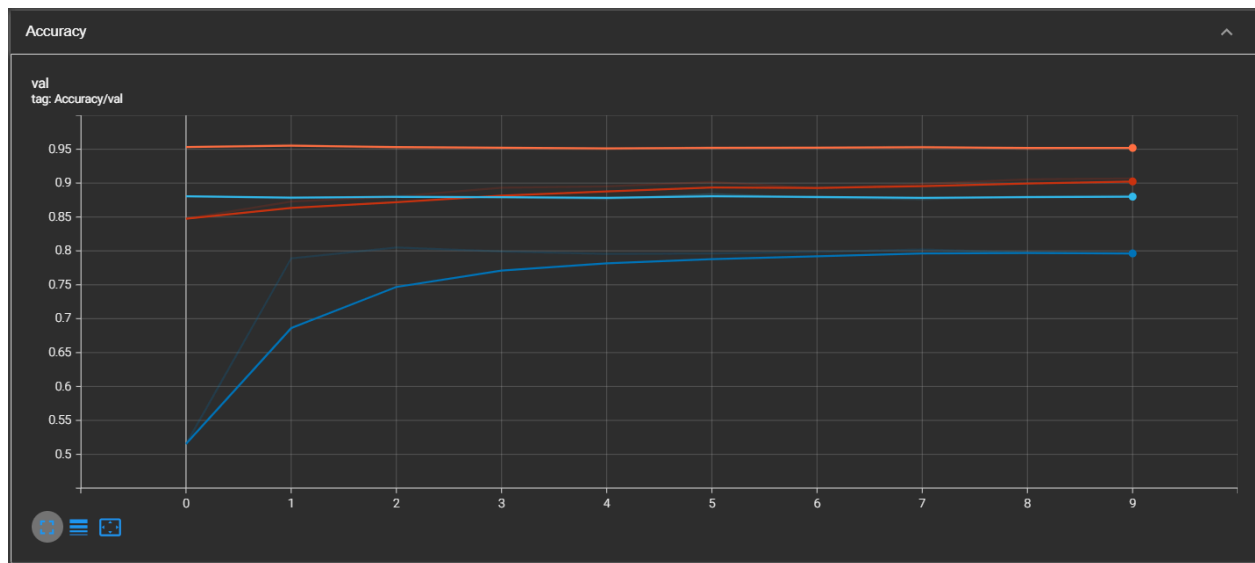
Confusion Matrix - IMDB LLaMa Model - Test Accuracy: 0.9485



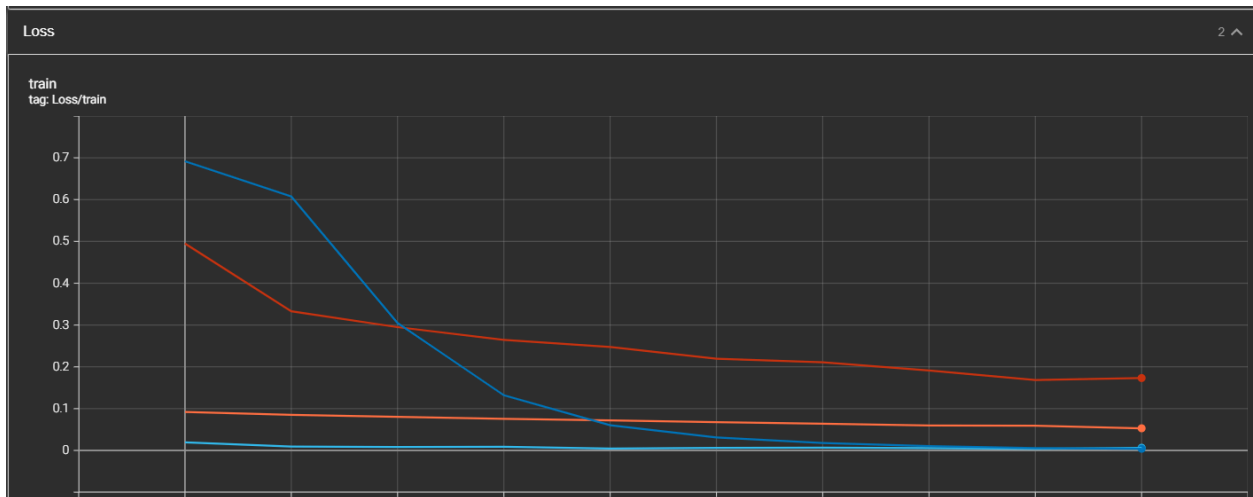
- Labels for the Curves



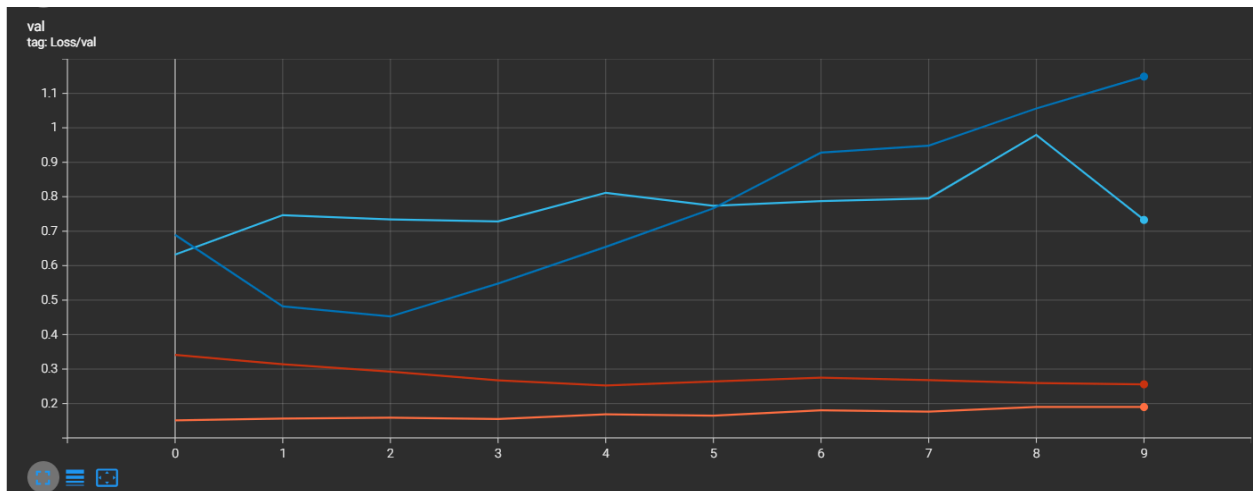
- Accuracy (Curve)



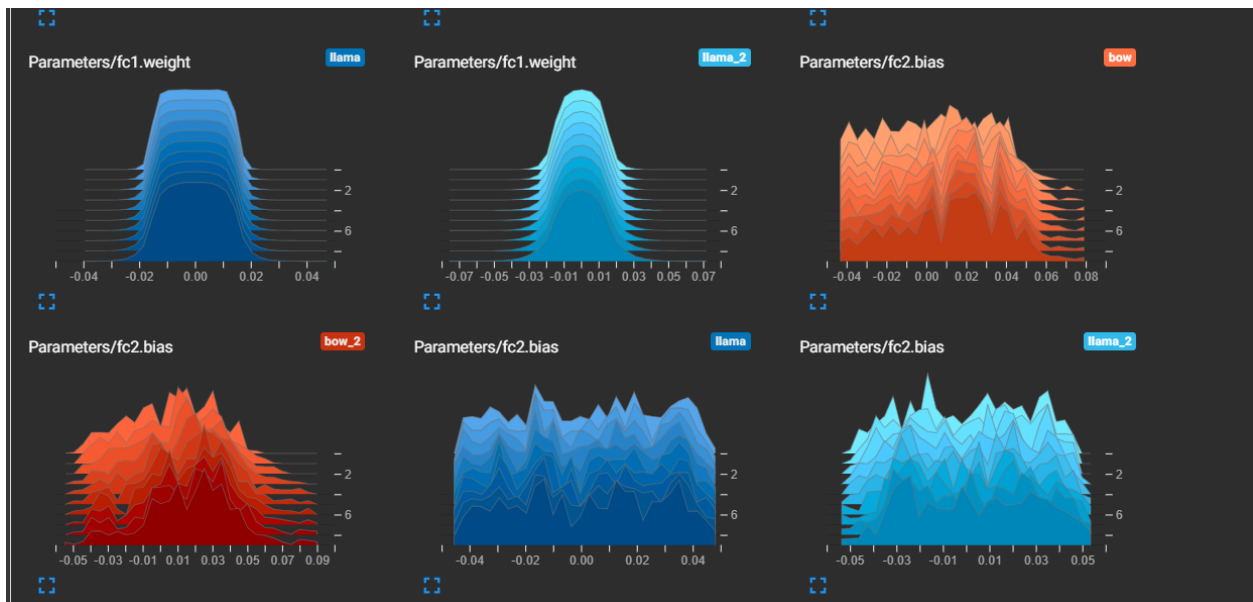
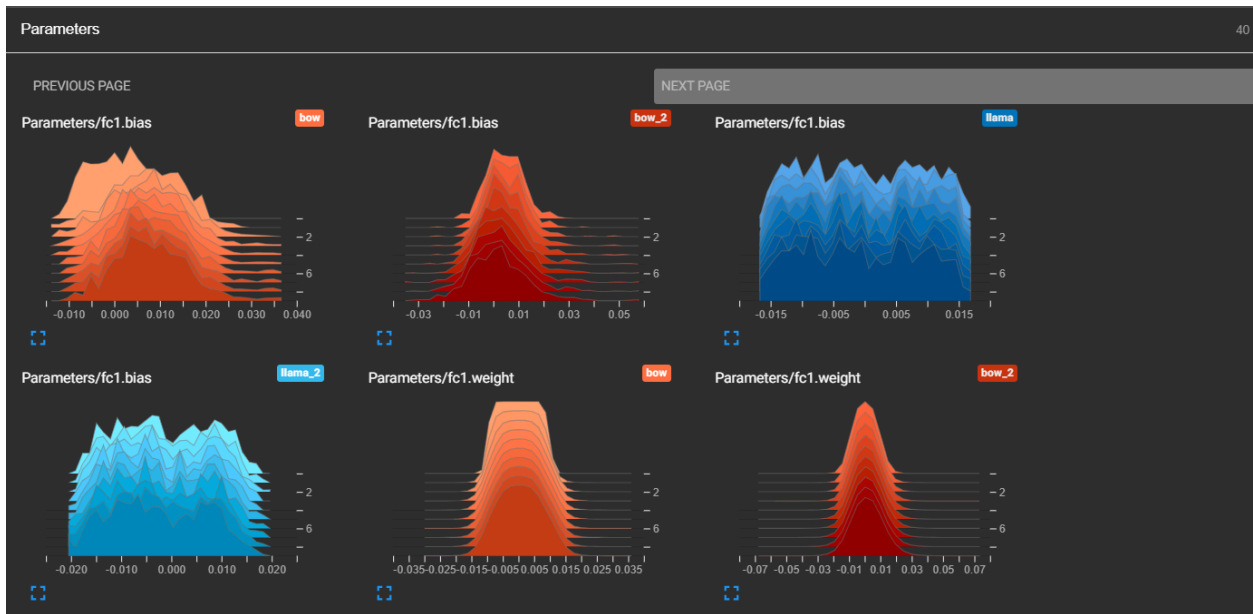
- Training Loss (Curve)



- Validation Loss (Curve)



- Parameters



Conclusion

BoW Model Performance

- On Dataset 1, the original BoW model (bow_1) achieved **79.41%** accuracy.
- On Dataset 2, the original BoW model (bow_1), achieved **77.33%** accuracy.

After continual learning (bow_2), the accuracy significantly increased to **88.65%**, demonstrating that training on the IMDB dataset helped the model better adapt to its linguistic patterns. This adaptation improved its ability to generalize, leading to enhanced test accuracy.

This improvement highlights how continual learning on the IMDB dataset allowed the model to better adapt to its specific vocabulary and sentiment structure, leading to enhanced generalization.

LLaMa Embedding Model Performance

- On Dataset 1, the LLaMa-based model (llama_1) performed exceptionally well, achieving **91.32%** accuracy, significantly outperforming BoW.
- On Dataset 2, the LLaMa-based model (llama_1) started with an accuracy of **91.72%** , significantly outperforming BoW.

After continual learning, (llama_2), the accuracy improved significantly to **94.85%**, demonstrating that additional training on the IMDB dataset enabled the model to better capture its linguistic patterns and sentiment structures. This improvement indicates that fine-tuning enhanced the model's ability to understand contextual nuances, leading to more accurate sentiment classification.

Model checkpointing served as the bridge between initial training and continual learning. By saving and restoring model states efficiently, it ensured that knowledge from Dataset 1 was not lost while adapting to Dataset 2. This allowed the models to build upon previous learning rather than starting from scratch, resulting in significant performance improvements after fine-tuning.

- Fine-tuning significantly boosts model performance, as seen in bow_2 and llama_2 on Dataset 2. However, this adaptation might introduce slight trade-offs in

generalization for unseen datasets.

- BoW models rely on word frequency, making them effective but limited in capturing deeper linguistic meaning and context.
- LLaMa embeddings, leveraging deep contextual understanding, outperform BoW models by capturing complex semantic relationships between words and sentences.